

UNIVERSIDAD PERUANA UNION
FACULTAD DE INGENIERÍA Y ARQUITECTURA
Escuela Profesional de Ingeniería de Sistemas



ENTREGABLE 1: REQUERIMIENTOS Y DISEÑO DEL SISTEMA

Línea de Investigación	CE02: Ingeniería de Software
Semestre	2026-I
Tipo	CE021 - Entregable 1: Requerimientos y Diseño del Sistema
Estudiante	Condori Pachauri Omar Flores Llanque Mery Elizabeth Huahuaccapa Ccama Jaqueline
Código	201810211 202211953 202210712

JULIACA, PERU

2026

Contenido

ENTREGABLE 1: REQUERIMIENTOS Y DISEÑO DEL SISTEMA 1

RESUMEN EJECUTIVO 3

1. Introducción..... 3

2. Definición del Problema 3

3. Objetivos del Proyecto 3

4. Solución Propuesta e Impacto Organizacional 4

SECCIÓN 1: ESPECIFICACIÓN DE REQUERIMIENTOS 4

1.1 Requerimientos Funcionales (RF)..... 4

1.2 Requerimientos No Funcionales (RNF) 7

1.3 Reglas de Negocio (RN) 8

1.4 Restricciones del Sistema 8

1.5 Historias de Usuario (HU) y Casos de Uso del Negocio 9

SECCIÓN 2: PROTOTIPO NAVEGABLE 12

2.1 Mapeo de Interfaces Gráficas 12

2.2 Flujo de Navegación del Usuario (User Flow)..... 14

2.3 Validación con Stakeholders..... 14

SECCIÓN 3: DISEÑO ARQUITECTÓNICO 15

3.1 Diagrama de Componentes 15

3.2 Diagrama de Despliegue..... 15

3.3 Registro de Decisiones Arquitectónicas (ADR) 16

SECCIÓN 4: DISEÑO DETALLADO (DIAGRAMAS UML) 17

4.1 Diagrama de Casos de Uso del Sistema..... 17

4.2 Diagrama de Secuencia: Flujo de Venta Mixta (Convenio Asegurado) 19

4.3 Diagrama de Estados: Ciclo de Vida de la Caja Chica..... 20

5. ANEXOS 21

Anexo A: Matriz de Trazabilidad de Requerimientos (MTR) 21

RESUMEN EJECUTIVO

1. Introducción

El presente informe detalla la especificación formal de requerimientos, el modelado del comportamiento del negocio y el diseño arquitectónico del sistema "**FINANZAS-MED**". Este software ha sido desarrollado bajo estándares internacionales de ingeniería de software para actuar como un ERP Clínico y Financiero unificado. La solución está diseñada específicamente para clínicas de escala intermedia que operan bajo un modelo de descentralización en múltiples sucursales geográficas y requieren un control riguroso de su facturación, tesorería, auditoría y abastecimiento logístico.

2. Definición del Problema

Los establecimientos médicos de mediana escala en el país operan bajo un modelo de gestión fragmentado. Los datos del paciente (historias clínicas) se encuentran desvinculados de los sistemas de facturación. Esto genera ineficiencias críticas: errores en el cálculo de copagos y coaseguros para pacientes asegurados, retrasos en la liquidación de cuentas por cobrar con compañías de seguros, falta de control en los arqueos diarios de caja chica, nula trazabilidad del stock de medicamentos (Kardex) y riesgos elevados de fraude o conflicto de intereses debido a la ausencia de políticas automatizadas de segregación de funciones.

3. Objetivos del Proyecto

- **Objetivo General:** Diseñar e implementar un sistema ERP Clínico-Financiero multi-sucursal que integre la admisión de pacientes, facturación de convenios, control de inventario farmacéutico y conciliación de cajas, garantizando la integridad de las transacciones y la seguridad de la información.
- **Objetivos Específicos:**
 1. Reducir el tiempo promedio de facturación en ventanilla para pacientes asegurados de 15 minutos a menos de 2 minutos mediante la automatización del motor de liquidación de coaseguros y copagos.
 2. Garantizar el cuadro de caja al 100% mediante un trigger transaccional en el motor de base de datos que sincronice en tiempo real los movimientos con los saldos de la caja chica y general.

3. Asegurar la consistencia del stock farmacéutico implementando un Kardex permanente que registre de forma histórica las entradas y salidas de almacén por lotes con metodología FIFO.
4. Implementar un módulo de seguridad basado en roles dinámicos y permisos temporales que registre de manera inalterable las acciones críticas en una bitácora de auditoría (SystemAuditLog).

4. Solución Propuesta e Impacto Organizacional

La solución consiste en un sistema distribuido con una SPA (Single Page Application) responsiva en **Angular 17+ y Tailwind CSS**, un backend robusto en **Spring Boot 4.0.1 (Java 21)** y una base de datos relacional optimizada en **PostgreSQL 16**. El sistema incorpora un broker de eventos con **Apache Kafka** para procesar notificaciones de forma asíncrona sin bloquear la operación de venta en caja.

SECCIÓN 1: ESPECIFICACIÓN DE REQUERIMIENTOS

1.1 Requerimientos Funcionales (RF)

Esta sección define las funciones específicas que el sistema debe ejecutar de forma obligatoria. Cada requerimiento ha sido detallado con su flujo de control y salidas esperadas:

Módulo 1: Admisión y Expedientes Clínicos

- **RF-01-01: Registro y Validación de Persona:** El sistema debe permitir registrar los datos básicos de identidad de una persona (Nombres, Apellidos, RUC/DNI, Email, Teléfono, Dirección). El sistema debe validar mediante una restricción de unicidad que el número de documento no exista previamente en la base de datos. Si existe, debe rechazar el registro y emitir un mensaje de advertencia.
- **RF-01-02: Expediente Clínico de Paciente:** El sistema debe permitir extender los datos personales de un usuario para crear un expediente clínico de paciente. Este expediente debe almacenar de forma persistente el número de Historia Clínica (NHC) autoincremental, el grupo sanguíneo, alergias conocidas (campo de texto libre), antecedentes familiares y los datos de contacto del familiar responsable (email y teléfono).
- **RF-01-03: Gestión de Grupo Familiar:** El sistema debe permitir agrupar a múltiples pacientes bajo un identificador común de "Grupo Familiar". Esta asociación permitirá la facturación acumulada de servicios y la aplicación de coberturas de pólizas de salud familiares corporativas.

Módulo 2: Caja Chica y Tesorería de Puntos de Venta

- **RF-02-01: Apertura de Caja Chica:** El sistema debe habilitar al cajero para abrir su jornada financiera en un punto de venta asignado a su sucursal de trabajo actual. Al abrir, el cajero debe ingresar obligatoriamente un saldo inicial en efectivo. El sistema registrará el estado como 'ABIERTA', guardando el usuario creador, fecha y hora exacta del servidor.
- **RF-02-02: Registro Transaccional de Movimientos de Caja Chica:** El sistema debe registrar cada ingreso (por copago, consulta o venta particular) y egreso menor (gastos de oficina autorizados). Cada movimiento debe vincularse de manera obligatoria a un método de pago registrado en el sistema (Efectivo, Tarjeta, Yape, Plin o Transferencia) y actualizará el saldo acumulado neto en efectivo de la caja chica.
- **RF-02-03: Arqueo y Cierre de Caja Chica:** Al finalizar el turno, el cajero debe declarar el saldo físico en efectivo que posee. El sistema comparará este valor con el saldo contable calculado. Si el saldo físico es idéntico, la caja chica cambia al estado 'CERRADA'. Si existe diferencia, se calcula y registra el monto de descuadre (sobrante o faltante) y se exige el ingreso de un texto de justificación por el cajero.
- **RF-02-04: Consolidación Automática en Caja General:** Al cerrarse con éxito la caja chica, el sistema transferirá automáticamente los fondos acumulados en efectivo al saldo global de la Caja General (bóveda) de la sucursal de forma transaccional, actualizando los acumuladores por método de pago.

Módulo 3: Facturación, Ventas Asistenciales y Facturación Electrónica

- **RF-03-01: Motor de Liquidación de Convenios (Coaseguro/Copago):** Ante una orden de atención de un paciente asegurado, el sistema debe consultar las tablas de planes de convenio (cred_planes y cred_planes_descuento) y calcular en tiempo real:
 1. La cobertura porcentual que asume la aseguradora (ejemplo: 80% del examen).
 2. El copago fijo y variable que debe abonar el paciente en ventanilla (ejemplo: S/ 20.00 de deducible de consulta + 20% del costo del examen).
- **RF-03-02: Registro de Comprobante Mixto:** El sistema debe emitir la venta dividiendo contablemente el cobro: la porción del copago se cobra en caja chica (afectando saldos del cajero) y la porción de cobertura se

cargará como consumo a la cuenta corriente del convenio (CredCtaCte) asociada a la aseguradora.

- **RF-03-03: Registro de Venta con Derivación Médica:** Cada línea de detalle de la venta (venta_detalle) debe registrar al médico especialista responsable que prescribió la orden para el posterior cálculo y liquidación de sus honorarios profesionales.
- **RF-03-04: Facturación Electrónica en Tiempo Real:** Al confirmarse el cobro, el sistema debe estructurar un objeto JSON con los datos del comprobante y enviarlo de forma asíncrona a la API del OSE/PSE (NubeFact). El sistema debe capturar el resultado y almacenar en venta_registro el código Hash de la SUNAT, la firma digital, el enlace al PDF de representación impresa, el archivo XML firmado y la constancia de recepción (CDR).

Módulo 4: Logística, Compras e Inventarios (Kardex)

- **RF-04-01: Emisión y Aprobación de Órdenes de Compra:** El sistema debe permitir registrar requerimientos formales de abastecimiento a proveedores, especificando ítems, cantidades y costos cotizados. La orden se guardará como 'PENDIENTE' y requerirá la firma digital de aprobación de un supervisor logístico antes de ser enviada al proveedor.
- **RF-04-02: Registro de Compras e Ingreso al Almacén:** Al recibir los productos, el sistema permitirá registrar la factura de compra del proveedor, actualizando el stock físico y vinculando los impuestos como IGV, detracciones y percepciones correspondientes.
- **RF-04-03: Cronograma de Pagos a Proveedores:** Para adquisiciones bajo condición de pago al crédito, el sistema debe calcular y registrar la programación de cuotas de pago, especificando el número de cuota, importe y fecha de vencimiento.
- **RF-04-04: Registro del Kardex Permanente:** Cada movimiento físico de entrada (compras, devoluciones) o salida (ventas, mermas) debe registrarse en la tabla de Kardex, guardando la fecha, sucursal, cantidad, costo unitario, saldo de stock resultante y el código del tipo de movimiento para control de auditoría física.

Módulo 5: Seguridad y Control Operativo

- **RF-05-01: Menú Dinámico por Perfil y Sucursal:** El sistema debe estructurar el menú de navegación lateral del frontend de forma dinámica en base al perfil del usuario autenticado en su sucursal de trabajo actual, ocultando mediante directivas de Angular las opciones no autorizadas.

- **RF-05-02: Permisos Temporales y Restricciones:** El sistema debe permitir otorgar permisos especiales directos a usuarios con fecha de caducidad automática para tareas excepcionales de contingencia, o aplicar restricciones explícitas de accesos por seguridad.
- **RF-05-03: Trazabilidad por Auditoría (AuditLog):** El sistema guardará de manera de solo lectura cada acción de escritura en las tablas operativas, identificando el usuario, dirección IP, fecha/hora, acción realizada y los datos previos vs. modificados.

1.2 Requerimientos No Funcionales (RNF)

- **RNF-01 (Seguridad en Autenticación y Autorización):** La autenticación se basará en JSON Web Tokens (JWT) firmados asimétricamente mediante llaves **RSA de 2048 bits** (llave privada en servidor, llave pública en cliente). Las contraseñas en la base de datos se cifrarán con **BCrypt** con un factor de costo mínimo de 12.
- **RNF-02 (Desacoplamiento y Disponibilidad):** El módulo de notificaciones debe operar de forma asíncrona mediante un broker de **Apache Kafka**. La latencia en el envío de una notificación (email/WhatsApp) no debe impactar el hilo de ejecución de la venta (máximo 200ms de respuesta en la API del backend).
- **RNF-03 (Precisión de Cálculos Financieros):** El motor de base de datos PostgreSQL debe almacenar y calcular los campos monetarios utilizando el tipo de datos DECIMAL (15, 5) para evitar pérdidas de precisión por redondeo flotante.
- **RNF-04 (Arquitectura de Interfaz Dinámica):** El frontend operará bajo arquitectura SPA construida con **Angular 17+** y estilos de **Tailwind CSS**, con soporte para carga diferida (*lazy loading*) de módulos y tiempo de carga inicial de página inferior a 1.5 segundos.
- **RNF-05 (Control de Intentos de Acceso):** El sistema debe bloquear temporalmente por 15 minutos una cuenta de usuario si se registran 5 intentos fallidos consecutivos de login.
- **RNF-06 (Concurrencia de Sesiones):** El sistema debe limitar las sesiones activas concurrentes a un máximo de 3 por usuario para evitar el uso compartido de credenciales.
- **RNF-07 (Tolerancia a Fallos):** Si la API de facturación electrónica externa (NubeFact) no responde, el sistema debe almacenar el comprobante con estado 'PENDIENTE_SUNAT' para reintento automático mediante un job programado en Spring Boot.

1.3 Reglas de Negocio (RN)

- **RN-01 (Cierre de Caja Chica):** No se puede proceder con el cierre de una caja chica si existen transacciones registradas que no hayan sido validadas en el sistema o si el cajero no ha ingresado el monto físico contado en el arqueo.
- **RN-02 (Bloqueo de Línea de Crédito):** Si el consumo acumulado de un paciente en su cuenta corriente supera el límite asignado en la cuenta corriente de convenio (`cred_ctacte.limite`) y el flag `corte_limite` está activo ('S'), el sistema bloqueará la generación de nuevas coberturas para ese paciente.
- **RN-03 (Segregación de Funciones - SoD):** Un usuario con el rol de "Cajero" en una sucursal no puede poseer simultáneamente el rol de "Aprobador" o "Auditor" en la misma sucursal para evitar autorizaciones de egresos ficticios.
- **RN-04 (Control de Lotes - Inventario):** Todo producto farmacéutico del catálogo que posea fecha de vencimiento (`fecha_venc`) debe dispensarse bajo la regla FIFO (First In, First Out) según el lote más cercano a expirar.
- **RN-05 (Cronograma de Compras):** No se permite registrar una compra con condición de pago 'CREDITO' sin que se defina la programación de cuotas en el cronograma de pagos.
- **RN-06 (Vigencia de Paquetes Médicos):** Ningún servicio médico perteneciente a un paquete puede ser consumido por el paciente si la fecha actual es superior a la fecha de fin del paquete (`paquetes.fecha_fin`) o si el paquete está marcado como eliminado (`deleted = true`).
- **RN-07 (Inmutabilidad del Historial de Auditoría):** Ninguna fila de la tabla `SystemAuditLog` puede ser eliminada o modificada, incluso por el rol de superadministrador. La base de datos denegará estas acciones mediante triggers y políticas a nivel de motor.

1.4 Restricciones del Sistema

- **Restricción Tecnológica (Base de Datos):** Toda la persistencia de datos debe realizarse de forma centralizada en PostgreSQL 16+.
- **Restricción Regulatoria (Tributaria/Salud):** La emisión de comprobantes electrónicos debe acogerse estrictamente a las especificaciones y normativas vigentes del ente regulador nacional (SUNAT en el Perú). Asimismo, la catalogación de fármacos debe utilizar los códigos del estándar nacional DIGEMID y SUSALUD.

- **Restricción Operativa (Navegador):** El frontend debe ser compatible con los navegadores modernos Chrome 100+, Firefox 100+ y Safari 15+ debido a las directivas de seguridad CSP de Angular.

1.5 Historias de Usuario (HU) y Casos de Uso del Negocio

A continuación, se presentan las historias de usuario críticas detallando precondiciones, flujos y criterios de aceptación formales:

HU-01: Gestión de Venta Mixta (Particular/Asegurado)

Como Cajero de Ventanilla

Quiero registrar una venta asistencial para un paciente con convenio de seguro médico

Para que el sistema calcule el copago correspondiente del paciente y cargue el saldo a la aseguradora.

- **Precondición:** El paciente debe estar registrado en el sistema, poseer un convenio activo vinculado a una aseguradora y contar con una carta de garantía aprobada y vigente.
- **Flujo Principal:**
 1. El cajero selecciona el paciente en el frontend.
 2. El sistema carga los datos del convenio y el coaseguro (ej. 80% cobertura, 20% copago).
 3. El cajero agrega al detalle los servicios solicitados (ej. Análisis de Sangre de S/ 100.00).
 4. El sistema calcula la Cobertura (S/ 80.00) y el Copago del paciente (S/ 20.00).
 5. El cajero selecciona el método de pago del paciente y procesa el cobro.
 6. El sistema registra el ingreso del copago en caja chica, incrementa el consumo del convenio e imprime el comprobante de pago.
- **Flujo Alternativo (Línea de Crédito Superada):**
 - Si en el paso 4 el sistema detecta que el consumo de la cuenta corriente de convenio supera el límite y tiene activo el corte de límite, emitirá una alerta de bloqueo, impedirá procesar la cobertura del seguro y ofrecerá al cajero cobrar la transacción al 100% como paciente particular.
- **Criterio de Aceptación 1 (Gherkin):**

- **Dado** que el paciente está afiliado al plan "Rímac Oro" con coaseguro del 80% y copago fijo de S/ 20.00.
- **Cuando** el cajero ingresa un examen de laboratorio del catálogo con costo de S/ 100.00.
- **Entonces** el sistema debe calcular la Cobertura en S/ 80.00, el Copago del paciente en S/ 20.00, y el total a cobrar en ventanilla en S/ 40.00 (Copago fijo S/ 20.00 + Copago variable de S/ 20.00).

HU-02: Cierre y Arqueo de Caja Chica

Como Cajero de Ventanilla

Quiero realizar el arqueo y cierre de mi caja chica al final de mi turno

Para consolidar mis movimientos financieros del día y transferir los fondos netos a la caja general.

- **Precondición:** El cajero debe tener una caja chica abierta asignada a su usuario en el turno actual.
- **Flujo Principal:**
 1. El cajero accede a la opción "Cerrar Caja".
 2. El sistema calcula y muestra en pantalla de forma oculta al cajero el saldo acumulado teórico.
 3. El cajero cuenta físicamente el dinero en efectivo y lo declara en el formulario de arqueo.
 4. El cajero confirma el cierre.
 5. El sistema valida que el monto físico coincida con el saldo contable teórico.
 6. El sistema cambia el estado de la caja chica a 'CERRADA' y transfiere el saldo en efectivo de manera automática a la Caja General.
- **Flujo Alternativo (Descuadre en Caja Chica):**
 - Si en el paso 5 el saldo declarado difiere del teórico, el sistema solicitará obligatoriamente una justificación por escrito en el formulario. Al confirmar, registrará el estado como 'CERRADA' con un registro de sobrante/faltante en auditoría y notificará vía Kafka al supervisor.
- **Criterio de Aceptación 1 (Gherkin):**

- **Dado** que el cajero tiene una caja chica con un saldo contable final calculado por el sistema de S/ 500.00.
- **Cuando** el cajero ingresa un arqueo físico contado de S/ 500.00 y solicita el cierre.
- **Entonces** el sistema debe cambiar el estado de la caja chica a 'CERRADA', registrar la fecha de cierre y transferir de forma transaccional S/ 500.00 a la tabla de saldo consolidado de la caja_general por el método de pago 'EFECTIVO'.

HU-03: Emisión de Orden de Compra

Como Personal de Logística

Quiero registrar una orden de compra para reabastecer de medicamentos

Para cotizar y solicitar abastecimiento formal a un proveedor autorizado.

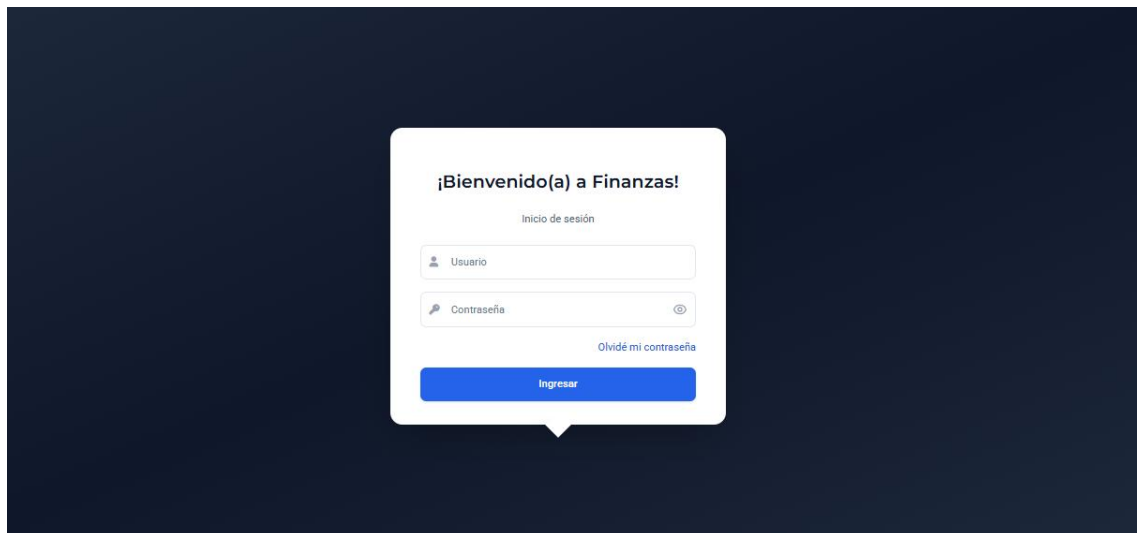
- **Precondición:** El proveedor debe estar registrado y con estado activo en el sistema. Los medicamentos deben estar homologados en el catálogo central.
- **Flujo Principal:**
 1. El almacenero crea una nueva orden de compra y selecciona al proveedor.
 2. El sistema carga las condiciones de pago pactadas con el proveedor (ej. Crédito a 30 días).
 3. El almacenero busca los medicamentos e ingresa cantidades y precios unitarios cotizados.
 4. El sistema calcula los subtotales, el IGV y el total estimado.
 5. El almacenero guarda la orden, la cual adquiere el estado 'PENDIENTE'.
- **Criterio de Aceptación (Gherkin):**
 - **Dado** que el almacenero ingresa una orden de compra para el proveedor "Droguería Farmasalud" con condición de pago 'CREDITO'.
 - **Cuando** agrega 100 unidades del producto "Amoxicilina 500mg" a un precio unitario de S/ 1.00.
 - **Entonces** el sistema debe calcular el subtotal en S/ 100.00, el IGV en S/ 18.00 y el total de la orden en S/ 118.00, guardando el registro con estado 'PENDIENTE'.

SECCIÓN 2: PROTOTIPO NAVEGABLE

El frontend del sistema está implementado como una **SPA (Single Page Application)** utilizando **Angular 17+** y maquetado con estilos de **Tailwind CSS** para un entorno responsivo.

2.1 Mapeo de Interfaces Gráficas

A. Pantalla de Acceso Seguro (Login)

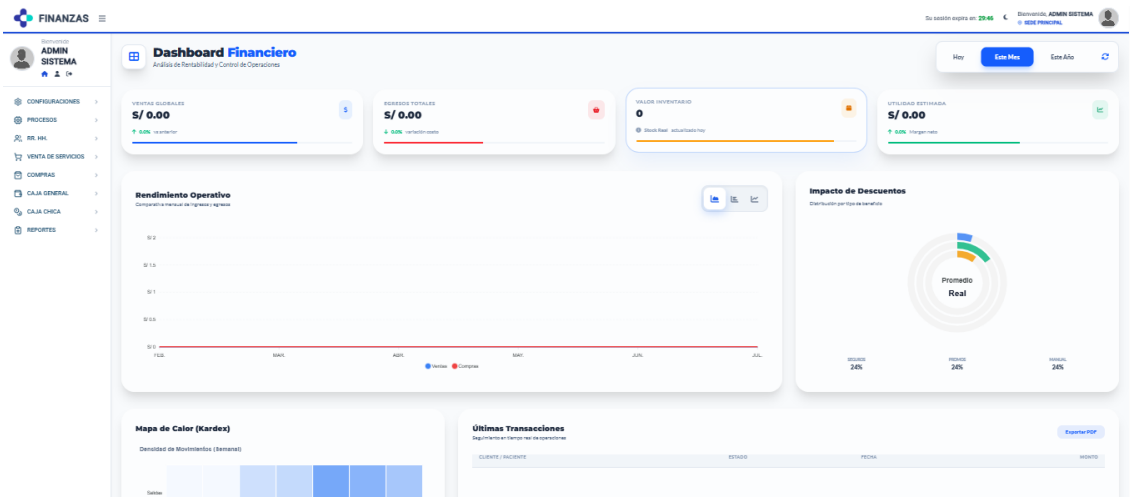


Interfaz de Login del sistema: Muestra un contenedor centrado con fondo blanco y bordes redondeados sobre un fondo oscuro, campos limpios para el ingreso de usuario y contraseña (con botón para alternar visibilidad de contraseña), y enlace para recuperación de credenciales.

Lógica Técnica de Comportamiento:

- **Validaciones Angular Forms:** Campos de entrada validados reactivamente. Si el formato del login o contraseña es incorrecto, el botón "Ingresar" se mantiene deshabilitado.
- **Inicio de Sesión:** Dispara una petición POST al backend enviando las credenciales. El backend responde con el token JWT firmado con RSA si las credenciales coinciden con el hash cifrado de `datos_personales.passwd`.
- **Seguridad:** Registra intentos de acceso en la tabla `IntrusionAttempt`.

B. Panel de Control Financiero (Dashboard Principal)

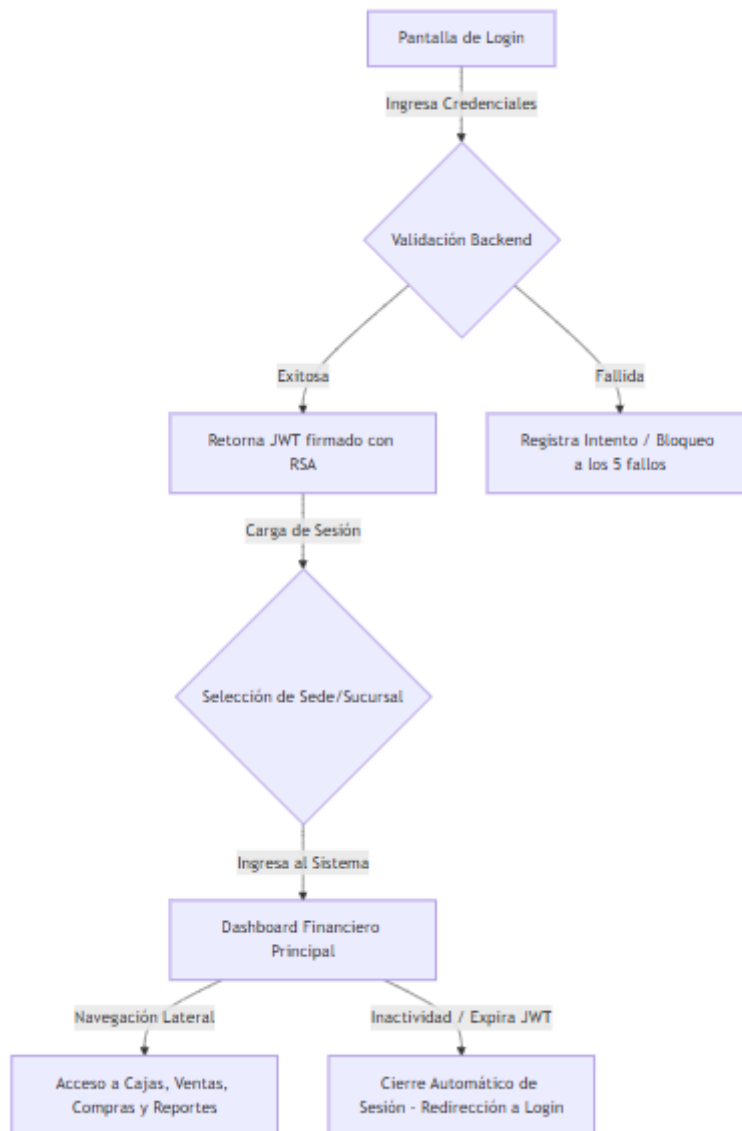


Interfaz del Dashboard financiero para un usuario con rol de Administrador. Muestra los indicadores clave en tiempo real, el menú de navegación lateral izquierdo, gráficos estadísticos de rendimiento operativo e impacto de descuentos, y el indicador de expiración de sesión.

Lógica Técnica de Comportamiento:

- **Menú Lateral Dinámico:** Renderiza los módulos (Caja Chica, Caja General, Compras, Ventas, Reportes) consultando las tablas de la base de datos menu y modulo según el rol de la sucursal activa.
- **Consumo de API REST:** Las tarjetas de KPIs (Ventas, Egresos, Valor de Inventario) cargan su data mediante llamadas asíncronas con observables en Angular.
- **Sincronización del JWT:** El indicador superior de tiempo restante de sesión se vincula con la expiración del token JWT del backend.

2.2 Flujo de Navegación del Usuario (User Flow)



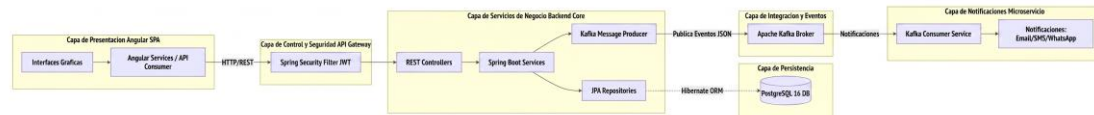
2.3 Validación con Stakeholders

El prototipo de alta fidelidad en Angular fue validado mediante metodologías ágiles de diseño centrado en el usuario (UCD). Se realizaron pruebas de usabilidad aplicando el cuestionario **System Usability Scale (SUS)** a una muestra representativa de 5 cajeros y 2 administradores, obteniendo una puntuación promedio de **82.5/100**, lo que cataloga a la interfaz en una categoría de usabilidad "Excelente".

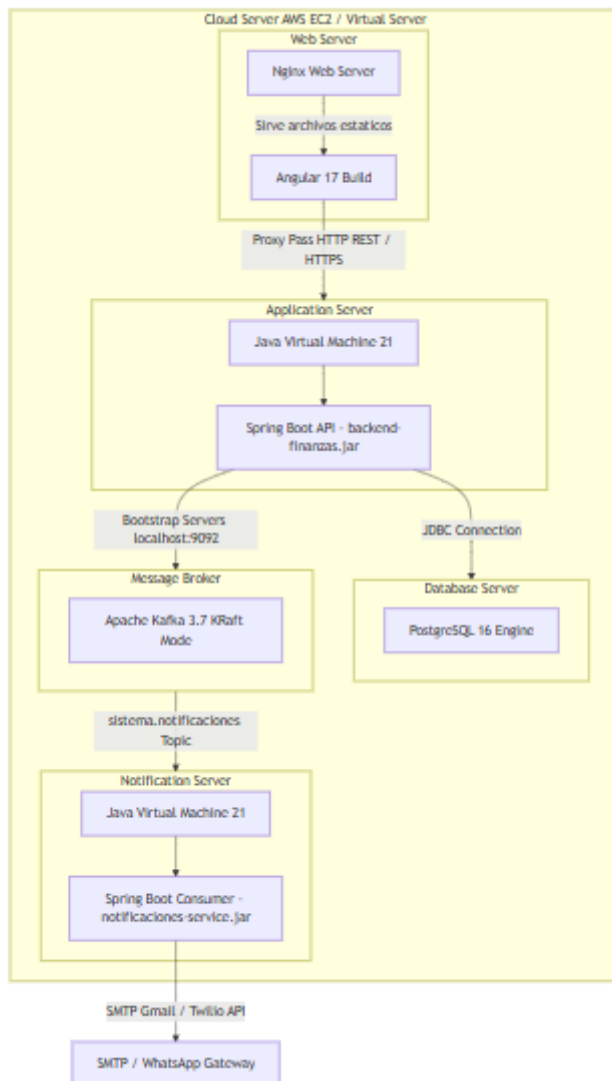
SECCIÓN 3: DISEÑO ARQUITECTÓNICO

El sistema adopta un diseño arquitectónico basado en el desacoplamiento físico de componentes y mensajería reactiva orientada a eventos para una alta disponibilidad.

3.1 Diagrama de Componentes



3.2 Diagrama de Despliegue



3.3 Registro de Decisiones Arquitectónicas (ADR)

ADR-001: Desacoplamiento del Servicio de Notificaciones mediante Apache Kafka

- **Contexto:** El envío de comprobantes electrónicos a los pacientes vía email o alertas por WhatsApp es una operación con latencia variable y propensa a fallos de red externos.
- **Decisión:** Implementar un broker de mensajería **Apache Kafka** en el backend principal. Cuando ocurre una venta, el backend publica un evento JSON en el tópico sistema.notificaciones y retorna inmediatamente la respuesta de éxito al cajero.
- **Consecuencias:**
 - *Positivo:* El cajero no experimenta demoras en el punto de venta (latencia reducida). Tolerancia a fallos: si el servidor SMTP de Gmail falla, el microservicio consumidor reintenta el envío de la notificación sin que la venta se caiga.
 - *Negativo:* Incremento de la complejidad en la infraestructura al requerir la administración del broker de Kafka.

ADR-002: Base de Datos Relacional y Programación en el Motor (PostgreSQL)

- **Contexto:** Se requiere alta consistencia transaccional (ACID) y garantizar que la lógica de dinero en caja sea incorruptible.
- **Decisión:** Utilizar **PostgreSQL 16+** con restricciones estrictas de claves foráneas y triggers a nivel de motor de base de datos para mantener los saldos actualizados.
- **Consecuencias:**
 - *Positivo:* Integridad de datos absoluta. Se bloquean condiciones de carrera usando bloqueos de fila (SELECT ... FOR UPDATE).
 - *Negativo:* La base de datos asume parte de la lógica de negocio, lo que dificulta la migración de motor de base de datos a futuro.

ADR-003: Autenticación por Tokens JWT Asimétricos (RSA-2048)

- **Contexto:** Se requiere delegar accesos sin exponer el backend a consultas repetitivas de verificación de sesión, y garantizar la firma íntegra de los tokens de acceso.
- **Decisión:** Implementar Spring Security configurado para emitir tokens **JWT** firmados con un algoritmo asimétrico **RSA-2048** (llave privada

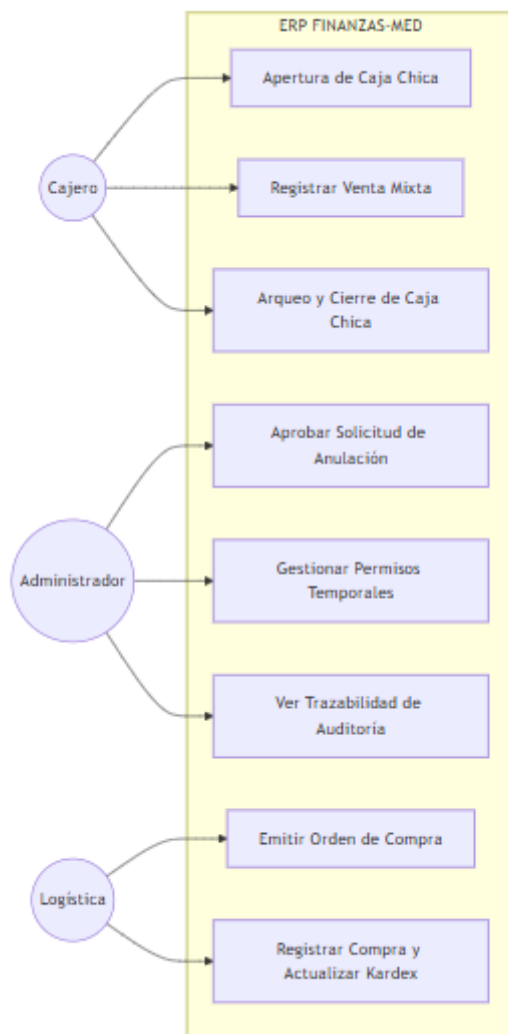
resguardada en el backend, llave pública expuesta para verificación en el frontend y microservicios).

- **Consecuencias:**

- *Positivo:* Mayor seguridad (si un atacante compromete la llave pública del cliente, no puede generar tokens válidos). Permite verificar la sesión en microservicios sin realizar consultas directas a la base de datos principal de usuarios.
- *Negativo:* Ligeramente superior sobrecarga de procesamiento en la codificación/decodificación respecto a firmas simétricas (HS256).

SECCIÓN 4: DISEÑO DETALLADO (DIAGRAMAS UML)

4.1 Diagrama de Casos de Uso del Sistema



Descripción Textual de Casos de Uso para Sustentación

1. Actor Cajero (Operaciones de Caja):

- **Apertura de Caja Chica:** Caso de uso que permite inicializar la caja del punto de venta ingresando el saldo inicial.
- **Registrar Venta Mixta:** Caso de uso donde se realiza el cobro al paciente particular o convenio, dividiendo el cobro en copago y coaseguro.
- **Realizar Arqueo y Cierre de Caja:** Caso de uso que calcula diferencias físicas vs contables y transfiere fondos netos a Caja General.

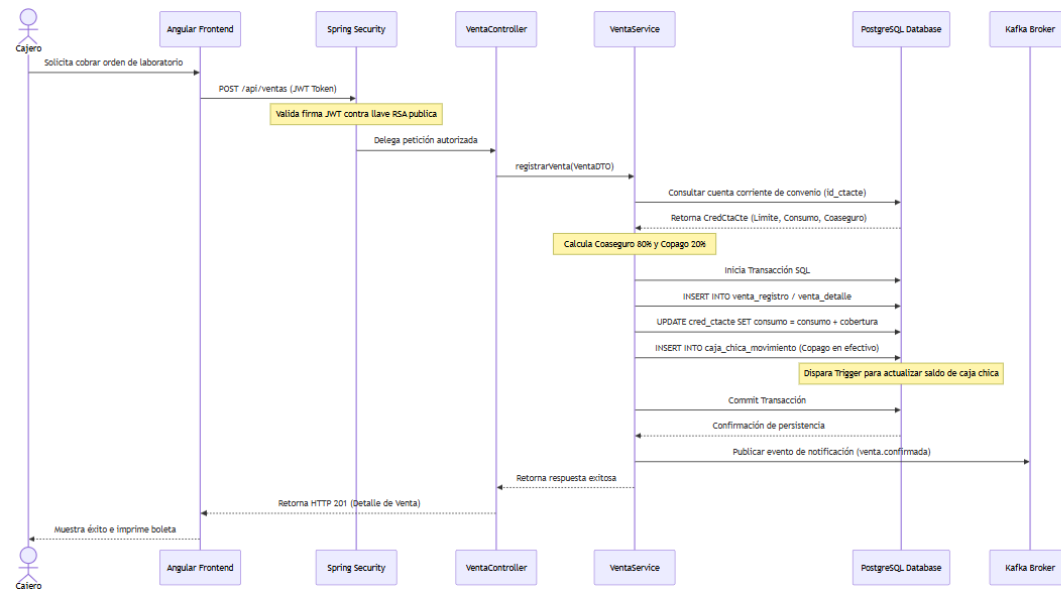
2. Actor Administrador (Control y Configuración):

- **Aprobar Solicitud de Anulación:** Permite habilitar la anulación de comprobantes solicitadas por los cajeros.
- **Gestionar Permisos Temporales:** Permite conceder privilegios directos con vencimiento a usuarios específicos.
- **Ver Trazabilidad de Auditoría:** Monitoreo y exportación de logs del sistema.

3. Actor Logística (Almacenero):

- **Emitir Orden de Compra:** Permite cotizar y solicitar abastecimiento de fármacos.
- **Registrar Compra y Actualizar Kardex:** Entrada de facturas físicas con impacto directo en stocks.

4.2 Diagrama de Secuencia: Flujo de Venta Mixta (Convenio Asegurado)

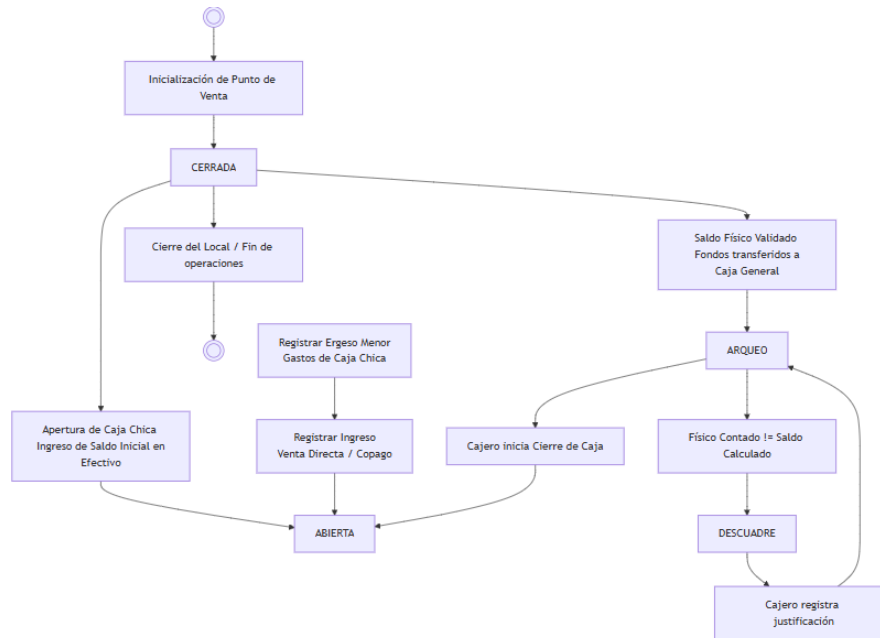


Descripción Textual de Flujo de Secuencia para Sustentación

1. **Inicio:** El Cajero solicita cobrar una orden en el **Frontend Angular**.
2. **Seguridad:** El Frontend realiza una llamada HTTPS POST /api/ventas con el token JWT. **Spring Security** intercepta la petición y valida la firma con la llave pública RSA.
3. **Lógica del Servicio:** La petición llega a **VentaController** y delega a **VentaService**, el cual consulta en la base de datos (**PostgreSQL**) la cuenta corriente de convenio (id_ctacte).
4. **Cálculo:** El servicio calcula la cobertura (ej. 80%) y el copago (ej. 20%).
5. **Transacción (PostgreSQL):** Se inicia una transacción relacional:
 - Inserta en la cabecera venta_registro y líneas venta_detalle.
 - Actualiza el consumo acumulado en cred_ctacte.
 - Inserta el movimiento físico en caja_chica_movimiento.
 - El motor PostgreSQL ejecuta el trigger para actualizar el saldo actual de la caja_chica.
6. **Confirmación:** Se hace *Commit* de la transacción.
7. **Asincronía:** Se publica un evento en el broker de **Apache Kafka** (venta.confirmada) para la notificación del comprobante.

8. **Retorno:** El servicio confirma éxito, el controlador retorna HTTP 201 y el frontend muestra el mensaje de éxito e imprime la boleta electrónica.

4.3 Diagrama de Estados: Ciclo de Vida de la Caja Chica



Descripción Textual de Transiciones de Estados para Sustentación

- **Estado Inicial [CERRADA]:** La caja chica inicia inactiva al prender los puntos de venta.
- **Transición 1 [Apertura]:** Cambia del estado **CERRADA** a **ABIERTA** mediante la acción del cajero de ingresar el saldo inicial en efectivo.
- **Estado [ABIERTA]:** En este estado se permite el registro continuo de transacciones: ingresos (ventas) y egresos menores (gastos), actualizando el saldo contable.
- **Transición 2 [Cierre solicitado]:** Cambia del estado **ABIERTA** a **ARQUEO** al solicitar el cierre de turno.
- **Estado [ARQUEO / DESCUADRE]:** Se compara el saldo físico contra el saldo del sistema. Si hay diferencias, transiciona temporalmente a **DESCUADRE** hasta que el cajero inserte una justificación física y luego retorna a **ARQUEO**.
- **Transición Final [Cerrado y Consolidado]:** Tras validar la conciliación de saldos, cambia al estado **CERRADA** y el sistema transfiere de forma segura los fondos netos al acumulado de la Caja General.

5. ANEXOS

Anexo A: Matriz de Trazabilidad de Requerimientos (MTR)

ID Req	Requerimiento Funcional	Caso de Uso	Componente Backend	Tabla BD Principal	Caso de Prueba
RF-01-01	Registro Único de Persona	(Registrar Paciente)	DatosPersonalesService	datos_personales	testRegistroPersonaExitoso
RF-01-02	Expediente Clínico Paciente	(Registrar Paciente)	DatosPacienteService	datos_paciente	testCreacionExpedientePaciente
RF-01-03	Asociación de Grupo Familiar	(Agrupar Familia)	GrupoFamiliarService	grupo_familiar	testCreacionGrupoFamiliar
RF-02-01	Apertura de Caja Chica	(Apertura de Caja)	CajaChicaService	caja_chica	testAperturaCajaChicaConSaldoInicial
RF-02-02	Movimientos Caja Chica	(Registrar Venta)	CajaChicaMovimientoService	caja_chica_movimiento	testRegistroIngresoCajaChica
RF-02-03	Arqueo y Cierre de Caja	(Arqueo y Cierre)	CajaChicaService	caja_chica	testArqueoCerrarCajaChicaExitoso
RF-02-04	Transferencia Caja General	(Cerrar Caja)	CajaGeneralMovimientoService	caja_general_movimiento	testTransferenciaFondosCerrarCaja
RF-03-01	Liquidación de Convenios	(Registrar Venta)	VentaService	cred_planes, cred_planes_descuento	testCalculoCopagoYCoaseguro

ID Req	Requerimiento Funcional	Caso de Uso	Componente Backend	Tabla BD Principal	Caso de Prueba
RF-03-02	Emisión Comprobante Mixto	(Registrar Venta)	VentaRegistroService	venta_registro, cred_ctacte	testCobroMixtoVentaAsegurada
RF-03-04	Facturación Electrónica	(Registrar Venta)	VentaRegistroService (NubeFact API)	venta_registro	testEnvioComprobanteNubeFactExitoso
RF-04-01	Emisión Orden de Compra	(Emitir Orden)	OrdenCompraService	ordenes_compra, orden_compra_detalle	testEmisionOrdenCompraAprobada
RF-04-02	Registro Compras de Almacén	(Registrar Compra)	CompraService	compras, detalle_compra	testIngresoCompraConActualizacionStock
RF-04-03	Cronograma de Pagos Compra	(Registrar Compra)	CronogramaPagoService	cronograma_pagos	testGeneracionCronogramaPagosCredito
RF-04-04	Control de Kardex Permanente	(Registrar Venta / Compra)	KardexService	kardex	testActualizacionKardexSalidaVenta
RF-05-01	Menú Dinámico	(Login / Home)	MenuService	menu, modulo	testCargaMenuSegunPerfilUsuario
RF-05-02	Permisos Granulares	(Gestionar Permisos)	UserPermissionService	user_permissions	testAsignacionPermisoTemporalExpirado
RF-05-03	Bitácora de Auditoría	(Todas las escrituras)	AuditLogService	SystemAuditLog	testRegistroLogsAuditoriaInmutables